Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Veritabanı Fiziksel Tasarımı ve Dosya Organizasyonu
  - Veritabanı tabloları arka planda diskte dosya olarak saklanır
  - Disk (HDD/SSD) ana hafızaya göre çok daha yavaştır (HDD: ~10^6 kat, SSD: ~100 kat)
  - Disk-ana hafızası iletişimi blok/sayfa (page) bazında gerçekleşir (tipik 4KB-32KB)
  - Büyük veri yığınlarında verinin düzenli saklanması ve hızlı erişimi için disk tabanlı veri yapıları kullanılır
- Dosya Organizasyonu Türleri
  - **HIP (Heap) Dosyası:** Kayıtlar rastgele veya append-only olarak yerleştirilir, sıralılık yoktur
  - **Sıralı Dosya:** Kayıtlar belirli bir niteliğe göre sıralanmış olarak tutulur
  - **Hash Dosyası:** Kayıtların yerleşimi bir hash fonksiyonuna göre belirlenir
  - **B-Tree Dosyası:** Kayıtlar ağaç yapısına uygun şekilde organize edilir
  - Dosya organizasyonu hem verinin yerleştirilmesiyle hem de erişimiyle ilgilidir
- Arama Problemi ve Çözüm Yaklaşımları
  - Sıralama yoksa tek çare sıralı tarama (sequential scan) - tüm dosyayı okumak gerekir
  - Sıralı dosyada binary search ile $\log_2 n$ adımda arama yapılabilir
  - Ancak disk tabanlı sistemlerde $\log_2 n$ yeterli değildir, B-Tree ile log_k(n) elde edilir
  - Hash ile tek adımda (O(1)) nokta sorguları çözülebilir
- Sorgu Türleri
  - **Nokta Sorgusu (Point Query):** Belirli bir değerin eşitliğini arar (equality search)
  - **Aralık Sorgusu (Range Query):** Belirli bir aralıktaki kayıtları arar
  - Hash: nokta sorgularında çok hızlı (tek adımda), ancak aralık sorgularında etkisiz
  - B-Tree: her iki sorgu türünde de etkilidir, aralık sorgularında yegane kullanılabilecek yapıdır

Hocanın Özellikle Vurguladığı Kısımlar

- B-Tree'nin her yerde kullanılan (ubiquitous) bir veri yapısı olduğu
  - Tüm veritabanlarında (SQL, NoSQL), işletim sistemlerinde, hatta cep telefonlarında kullanılır
  - Veritabanlarının hızlı çalışmasının temelinde B-Tree'nin yattığı
- Disk erişimi ile ana hafıza erişimi arasındaki uçurum
  - Disk erişimi milisaniye mertebesinde, ana hafızası nanosaniye mertebesinde
  - Index kullanımıyla 100 binlerce kat hız artışı sağlanabilir
  - Bu hızlanma olmadan gerçek zamanlı veri tabanı işlemi mümkün değildir
- Index'in avantajları vedezavantajları
  - **Avantaj:** Sorguları 100 binlerce kat hızlandırır
  - **Dezavantaj 1:** Yer kaybı (yaklaşık nitelik sayısının 1/10'u kadar)
  - **Dezavantaj 2:** Veri tekrarı → veri bütünlüğü tehdidi, tutarsızlık riski
  - **Dezavantaj 3:** Ekleme/silme işlemlerinde ek yük (update propagation)
- Primary Index ve Secondary Index farkı
  - **Primary Index:** Primary key (benzersiz nitelik) üzerine oluşturulur
  - **Secondary Index:** Primary key olmayan nitelikler üzerine oluşturulur
  - Secondary index'te aynı değer birden fazla kayda işaret edebilir (çoklu RID listesi)

- Index seçiciliği (selectivity) önemli
  - Düşük seçicilikte (örn. %90'ı aynı değer) index kullanmaktan vazgeçilebilir
  - Optimizer katalogdaki istatistiksel bilgilere göre index kullanıp kullanmamaya karar verir
  - Bazen index olmasa bile tablo taraması daha hızlı olabilir

Kısa Tekrar Notları

- B-Tree arama maliyeti: log_k(n) disk erişimi (k: fan-out, tipik 200-500)
- 300 milyon kayıtta B-Tree yüksekliği 4 ise sadece 3-4 disk erişimi yeterli
- Tampon (buffer/cache)sayesinde kök ve yakın düğümler ana hafızada tutulur, fiili erişim 1-2'ye düşer
- B-Tree dengeli bir ağaçtır; kökten yapraklara uzaklık her zaman aynıdır
- Eklemelerde split (bölme), silmelerde merge (birleştirme) işlemi yapılır
- Root split olduğunda ağacın yüksekliği artar
- %67 doluluk oranı empirik olarak yaygındır; bulk loading ile %100'e yaklaştırılabilir
- Disk sayfası başına KP (key-pointer) çift sayısı sayfanın boyutuna ve anahtar uzunluğuna bağlıdır
- 4KB sayfada tipik 200-500 KP çifti sığar

Detaylı Açıklamalar

Index Yapısı ve Oluşturma

Index, veri dosyasını sıralamaya gerek kalmadan düzenli erişim sağlayan yardımcı dosyadır. Bir nitelik (örn. label, composer, title) üzerine oluşturulacak index için: 1. Ana dosyadan o niteliğin tüm değerleri alınır 2. Bu değerler sıralanır 3. Her değerin yanına ana dosyadaki offset (kayıt adresi) yazılır 4. Oluşan bu küçük dosya "index dosyası"dır

Index dosyası, ana dosyanın yaklaşık 1/10'u kadar yer kaplar (tek nitelik + offset adresi içerdiği için).

B-Tree (B+Tree) Yapısı

B+Tree, disk tabanlı en yaygın ağaç veri yapısıdır. Temel özellikleri: - **Yaprak düğümler:** Tüm veri kayıtları (veya RID'ler) yapraklarda bulunur ve birbirine link list ile bağlıdır - **İç düğümler:** Sadece arama anahtarları ve pointer'lar içerir (index yapısını oluşturur) - **Sıralılık:** Her düğüm içindeki anahtarlar sıralıdır; sol tarafta küçük eşit, sağ tarafta büyük eşit değerler bulunur - **Denge:** Ağaç her zaman dengelidir; yapraklara uzaklık hep aynıdır - **Order (d):** Her düğümün en az d, en fazla 2d anahtar-Pointer çifti içerebilir - **Fan-out (k):** Bir disk sayfasından çıkabilecek pointer sayısı (tipik 200-500)

B-Tree'de Arama

Arama her zaman root'tan başlar. Her düğümde local binary search yapılarak doğru dala gidilir. Toplam maliyet: log_k(n) disk erişimi. Örnek: 300 milyon kayıt, k=133, yükseklik=4 → 4 disk erişimi; tampon kullanımıyla 1-2 disk erişimi.

B-Tree'de Ekleme (Insert)

1. Root'tan başlayarak eklenecek kaydın bulunacağı yaprak düğüm tespit edilir
2. Eğer yaprakta yer varsa kayda sıralı olarak eklenir
3. Eğer yaprak doluysa iki seçenek vardır:
   - **Redistribute (Yeniden Dağıtma):** Komşu yaprakta yer varsa kaydı oraya aktar ve üst düğümdeki ayırıcı anahtarı güncelle
   - **Split (Bölme):** Yaprak sayfasını ikiye böl, ortadaki anahtarı üst düğüme ekle
4. Eğer üst düğüm de doluysa split yukarıya doğru zincirleme devam eder
5. Root split olduğunda yeni bir root oluşturulur ve ağacın yüksekliği 1 artar

B-Tree'de Silme (Delete)

1. Silinecek kaydı yaprakta bul ve sil
2. Eğer yapraktaki doluluk oranı alt sınırın (d) altına düşerse:
    - **Redistribute:** Komşu yapraktan ödünç al
    - **Merge (Birleştirme):** İki yaprağı birleştir
3. Merge işlemi üst düğümlere kadar yayılabilir
4. Root'un iki çocuğu merge olduğunda root düşer ve ağacın yüksekliği 1 azalır
5. Sistemler genellikle merge işlemlerini erteler (lazy deletion); periyodik bakım ile yapı yeniden düzenlenir

B-Tree'nin Yer Verimliliği

- En kötü durum: %50 (tüm sayfalar yarı dolu)
- Ortalama durum: %67-70 civarı
- Bulk loading (toplu yükleme) ile veri önceden sıralanıp yüklendikten sonra %100'e yakın doluluk elde edilebilir
- Split işlemleri yer verimliliğini düşürür; merge ise artırır

Multi-Dimensional Index

- Birçok nitelik üzerine aynı anda index oluşturulabilir
- Uzamsal (spatial) ve çok boyutlu (multi-dimensional) index yapıları vardır
- Bu konu hem klasik hem de güncel araştırma konusudur
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.